

## 22 | 自定义组件：如何满足业务的个性化需求？

2022-5-18 况众文 朴惠姝



你好，我是众文，这一讲还是由我和惠姝来讲解。

上一讲，我们讲了如何构建混合应用。当环境配置、载体页、调试打包都 OK 后，我们就要开始复杂业务的开发了。在实际开发中，除了负责 React Native 框架本身的维护迭代外，另一个重要的工作就是配合前端业务，开发对应的 Native 组件。

那么什么时候用这些自定义的 Native 组件呢？

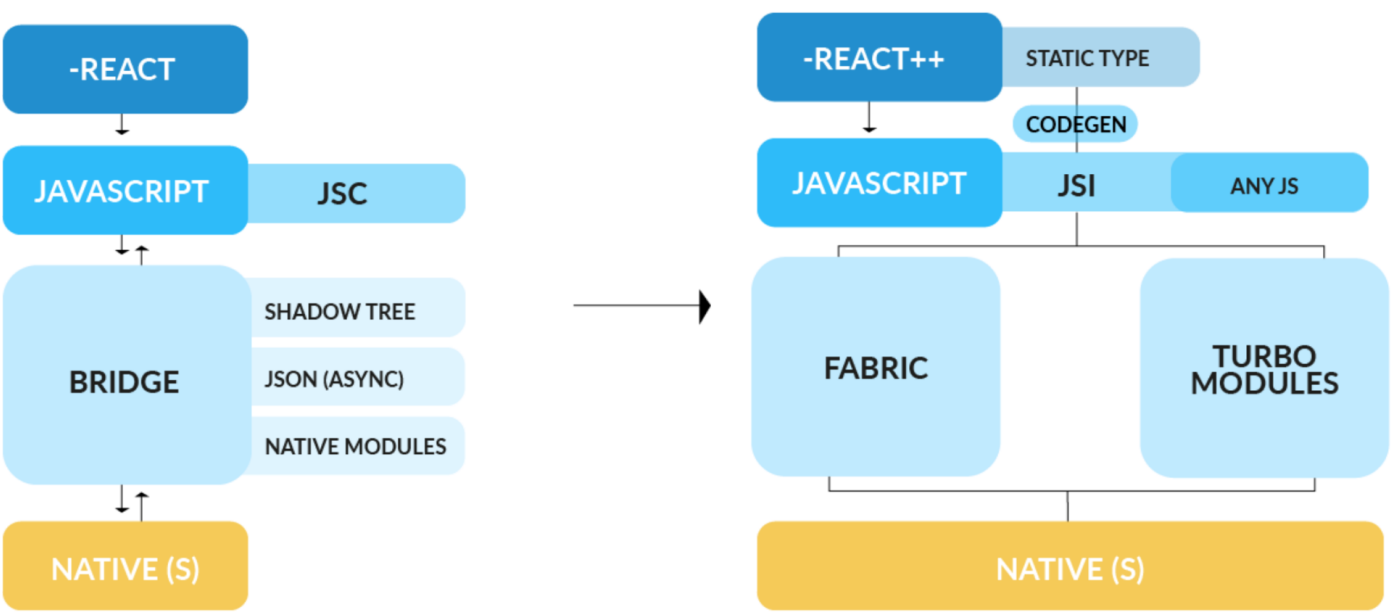
比如，有时候 App 需要访问平台 API，但 React Native 可能还没有相应的模块包装；或者你需要复用公司内的一些用 Java/OC 写的通用组件，而不是用 JavaScript 重新实现一遍；又或者你需要实现某些高性能的、多线程的代码，譬如图片处理、数据库，或者各种高级扩展等。

当然，你可以通过官方文档（[Android](#)/[iOS](#)），快速访问你的原生模块。但官方文档提供的主要是简单的 Demo 和步骤，在实际开发中，你可能还需要自定义组件的方方面面，包括新架构定义组件的全流程，以及实际业务中的踩坑指南等。

今天这一讲，我们会先带你补齐组件的相关基础知识，包括组件的生命周期、组件传输数据类型，并以新架构的TurboModule 和 Fabric 为案例，带你了解自定义组件的方方面面。你也能借此对React Native新架构建立起初步认识。接下来让我们先了解下期待已久的 React Native 新架构。

## 新架构介绍

我们现在先通过下面这张图简单对比下新老架构：



新架构变更点主要在下面这几个方面：

| #             | 说明                                                                                                                                                            |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 通信            | JSI 替换 bridge，性能大幅提升，JSI 无需异步序列化与反序列化过程，可实现同步的 JavaScript 和原生 Java 或者 OC 的通讯。同时原有 bridge 数据通讯采用 JSON 和基础类型数据，而 JSI 还支持 JSI Object                             |
| JavaScript 引擎 | JSI 作为 JSC 之上的抽象，JSI 用来屏蔽 JavaScript 引擎的差异，允许换用不同的 JS 引擎 (JSC、V8、Hermes)                                                                                      |
| Fabric        | 对标旧框架的 UIManager，FabricUIManager 可以和 C++ 层直接进行通讯，解除了原有的 UIManager 依赖单个 bridge 的问题，有了 JSI 后，以前批量依赖 bridge 的 UI 操作，都可以同步的执行到 C++ 层，性能得到大幅提升，特别是在列表快速滑动、复杂动画交互方面 |
| TurboModules  | 旧框架 Native Modules 需要在启动时全部初始化，影响启动速度，TurboModules 允许按需加载 Native 模块，并在模块初始化之后直接持有其引用，不再依靠消息通信来调用模块功能                                                          |
| CodeGen       | 新架构 UI 增加了 C++ 层的 shadow、component 层，而且大部分组件都是基于 JSI，因而开发 UI 组件和 API 的流程更复杂了，要求开发者具有 C++、JNI 的编程能力，为了方便开发者快速开发 Facebook 也提供了 codegen 工具，帮助生成一些自动化的代码          |



前面也说了，我们一讲将以 TurboModule 和 Fabric 为案例对自定义组件进行讲解。所以你现在需要对新架构有一个初步的认识，特别是要注意，TurboModule 和 Fabric 对比旧版的 Native Module 和 UIManager 有哪些差异和优势。

如果你还想了解新架构的更多信息，你可以参考[官方文档](#)。这里包括了新架构介绍、如何在 Android、iOS 上开启新架构、如何在 Android、iOS 上开启使用 TurboModule 和 Fabric 等。你也可以根据官方文档试着编译运行新架构。

而且，[react-native-screens](#)、[react-native-gesture-handler](#) 等知名 React Native 库的新版都已适配了新架构，感兴趣的话，你可以去了解下。

好的，现在我们进入这一讲的正题，先来了解一下组件的生命周期。

## 组件的生命周期

组件的生命周期，指的是在组件创建、更新、销毁的过程中伴随的各种各样的函数执行。这些在组件特定的时期被触发的函数，统称为组件的生命周期。

让组件拥有生命周期，我们就可以更好地管理组件的状态、内存，跟随载体页的生命周期做相应的处理。

# Android

在Android端，一个 Module 组件的生命周期包括：

构造 -> 初始化 -> onHostResume() -> onHostPause() -> onHostDestroy()

这几个生命周期的意思是：

|                 |                                                                                                                                                                      |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 构造              | 组件创建。这里需要注意：旧版本的 <b>NativeModule</b> 是在载体页初始化 React Native 环境时，全部进行初始化创建的；而 <b>TurboModule</b> 则不同，TurboModule 采用懒加载模式，在 JavaScript 运行时第一次 import 该TurboModule 时进行创建 |
| initialize      | 初始化                                                                                                                                                                  |
| onHostResume()  | 载体页 onResume() 时触发，一般是指载体页与用户能进行交互时被执行                                                                                                                               |
| onHostPause()   | 载体页 onPause() 时触发，一般是载体页被弹窗遮挡，或者即将离开前也会触发                                                                                                                            |
| onHostDestroy() | 载体页销毁时                                                                                                                                                               |



如果你不熟悉 Android Activity/Fragment 生命周期，那你可以看下[这篇文章](#)加深下理解。

那么如何让 Native Module 具备生命周期呢？

React Native 给我们提供了一个接口：

com.facebook.react.bridge.LifecycleEventListener。我们只需要在组件中添加这个接口的注册和取消注册，就可以让组件具备生命周期了。这里要注意，不要忘记在 onHostDestroy() 中移除注册，否则会造成内存泄漏。示例代码如下：

```

public class TestJavaModule extends ReactContextBaseJavaModule
implements LifecycleEventListener, ReactModuleWithSpec, TurboModule {
    public TestJavaModule(ReactApplicationContext reactContext) {
        super(reactContext.real());
        reactContext.addLifecycleEventListener(this);
    }

    @Override
    public String getName() {
        return getClass().getSimpleName();
    }

    @Override
    public void onHostResume() {
    }

    @Override
    public void onHostPause() {
    }

    @Override
    public void onHostDestroy() {
        getReactApplicationContext().removeLifecycleEventListener(this);
    }
}

```

其实组件的生命周期原理很简单，就是观察者模式。当载体页触发自身的生命周期回调时，调用 `ReactInstanceManager` 的 `onHostXXX()` 方法，`ReactInstanceManager` 进而调用 `ReactContext` 的对应回调。

比如载体页调用 `onResume()` 时，最终会调用 `ReactContext` 的 `onHostResume()`，内部会遍历注册的事件进行回调：

```

public void onHostResume(@Nullable Activity activity) {
    Iterator iterator = this.mLifecycleEventListeners.iterator();
    while(iterator.hasNext()) {
        LifecycleEventListener listener = (LifecycleEventListener) iterator.next();
        // 观察者模式，载体页时调用已注册组件的生命周期回调
        listener.onHostResume();
    }
}

```

以上就是 Android 端组件生命周期的讲解，我们再来看看 iOS 端。

## iOS

iOS中 NativeModules 组件的创建销毁时机，与bridge的创建销毁时机完全一致：

alloc：创建当前组件；

dealloc：销毁当前组件。

创建一个组件TestNativeModule，通过RCT\_EXPORT\_MODULE() 声明组件，默认会根据类名声明组件名，当然也可以通过在参数中传入其他字符串作为组件的名。

```
@implementation TestNativeModule
RCT_EXPORT_MODULE()

- (instancetype)init{
    self = [super init];
    return self;
}

- (void)dealloc{
    NSLog(@"dealloc");
}
```

而 **TurboModule** 组件的生命周期却与 NativeModule 不同。TurboModule 采用懒加载模式，在 Bridge 创建后页面中第一次 import 当前TurboModule ，也就是 JavaScript 端通过 **TurboModuleRegistry.getEnforcing**方法加载组件时， Native 会创建对应的 TurboModule 并进行缓存。如果 JS 端没有加载当前自定义组件，该组件就不会进行初始化。

JS 端加载组件方式如下：

```
export default (TurboModuleRegistry.getEnforcing<Spec>(
    'TestTurboModule')
    : Spec);
```

而TurboModule 的销毁时机与 Bridge 的销毁时机一致。Bridge 进行销毁时会发送一个RCTBridgeDidInvalidateModulesNotification 通知，TurboModuleManager会监听该事件，依次对所有已创建的 TurboModule 进行销毁。示例代码如下：

```
- (void)bridgeDidInvalidateModules:(NSNotification *)notification
{
    RCTBridge *bridge = notification.userInfo[@"bridge"];
    if (bridge != _bridge) {
        return;
    }
    [self _invalidateModules]; //销毁所有TurboModules
}
```

了解完了组件的生命周期，我们再来看下组件的传输数据类型。在组件运行过程中，Native 与 JavaScript 不可避免地需要进行数据交互，如 JavaScript 调用组件方法传入数据，Native 向 JavaScript 回传结果，而 React Native 也帮我们封装好了对应的数据类型。

## 组件传输数据类型

在 Native 与 JavaScript 通信的过程中，组件需要获取输入参数、回传结果，对此 React Native 给我们包装了相应的数据类型，方便快速操作，我们通过一个Demo来简单了解下。

现在，我们让JavaScript端调用 TestModule 的 testMethod 方法，传入参数 type 和 message，接收 native 回传数据：

```
NativeModules.TestModule.testMethod({type: 1, message: "fromJS"}, (result)=>{
    console.info(result);
});
```

然后我们来看TestModule 在 Android、iOS 侧的实现。先来看 Android 端是怎么做的。

不过，在实现 TestModule 之前，我们需要先了解下 Android 端的组件传输数据类型：

| JavaScript 端 | Android 端 |
|--------------|-----------|
| Bool类型       | boolean   |
| double类型     | double    |
| int类型        | int       |
| String类型     | String    |
| Array类型      | Array     |
| Object类型     | Map       |



这里你要注意，数字类型有点特殊。因为 JavaScript 不支持 long 64 位长类型，只支持 int (32)和double，所以对于长数字，JavaScript端统一用 double 表示。那么 Android 端如何转换成自己需要的数据类型呢？

我们以 long 为例，可以这样参考[官方issue](#)这样处理：



```

double value = readableMap.getDouble(key);
try {
    // 判断是否为 long 的范围：超过了 int 的最大值且为整数
    if (value > Integer.MAX_VALUE && value % 1 == 0) {
        long cv = (long) value;
        // 转换成 long 型返回
    }
} catch (Exception e) {
    // 异常时，仍使用 double
}

```

这段代码中，我们先将 JavaScript 传入的数值统一以双精度浮点数 double 来获取。获取完后，判断这个值是否超出了整数的最大值且不为小数，条件符合就将它转换成长整数 long，否则还是以 double 来返回。

了解完 Android 端的组件数据传输类型后，我们就可以来实现上文中的 TestModule 了：

```

public class TestModule extends ReactContextBaseJavaModule implements ReactModuleWithSpec, T {
    public TestModule(ReactApplicationContext reactContext) {
        super(reactContext.real());
    }

    @Override
    public String getName() {
        return getClass().getSimpleName();
    }

    @ReactMethod
    public void testMethod(ReadableMap data, Callback callback) {
        // 获取 JS 的调用输入参数
        int type = data.getInt("type");
        String message = data.getString("message");
        // 回传数据给 JS
        WritableMap resultMap = new WritableNativeMap();
        resultMap.putInt("code", 1);
        resultMap.putString("message", "success");
        callback.invoke(resultMap);
    }
}

```

上面代码中，我们定义了 Native 组件 TestModule，内部实现了 JavaScript 需要调用的 testMethod 方法。此方法包含两个参数：ReadableMap 和 Callback。ReadableMap 为 JavaScript 传入参数的字典，我们可以通过对应的 key 获取到 JavaScript 的入参值，而 Callback 是在 Native 回传数据时需要使用的，后面我们还有对通信方式这部分的讲解，这里我们只需要了解一下就好。

具体实现上，我们是首先获取 JavaScript 调用传入的 type 和 message，然后再通过 WritableMap 写入数据，最后通过 callback 回传给 JavaScript。

以上就是 Android 端的 React Native 读写数据类型，我们再来看下 iOS 端。

iOS端也是一样，在实现前面这个 demo 前，我们需要先看下 iOS 端支持的传入数据类型：

| JavaScript 端 | iOS 端          |
|--------------|----------------|
| Bool类型       | NSNumber对象     |
| double类型     | NSNumber对象     |
| String类型     | NSString对象     |
| Array类型      | NSArray对象      |
| Object类型     | NSDictionary对象 |



那么，iOS端中是如何实现上文中的TestModule的呢？我们可以在Module中进行callback，然后通过NSArray来返回，如下：

```
RCT_EXPORT_METHOD(getValueWithCallback : (RCTResponseSenderBlock)callback){  
    if (!callback) {
```

```
    return;  
  }  
  callback@[ @"value from callback!" ] );  
}
```

不过，我们这个组件案例，只是演示了 Native 可以通过 callback 向 JavaScript 回传数据。那么除了 callback，React Native 与原生还有什么通信方式呢？

## React Native 与原生的通信方式

总体来说，native 向 JavaScript 传递数据的方式分成以下三种：

Callback：由 JavaScript 主导触发，Native 进行回传，一次触发只能传递一次；

Promise：由 JavaScript 主导触发，Native 进行回传，一次触发只能传递一次。Promise 是 ES6 的新特性，类似 RXJava 的链式调用。Promise 有三种状态，分别是 pending（进行时）、resolve（已完成）、reject（已失败）；

发送事件：由 Native 主导触发，可传递多次，类似 Android 的广播和 iOS 的通知中心。

Callback 在上面的例子中已经出现过了，我们通过 `callback.invoke(xx)` 就可以将数据回传给 JavaScript，使用起来比较简单，这边我们就不再赘述了。现在我们主要来看下 Promise 和发送事件的示例，以便更好地了解 React Native 和原生之间是如何进行通信的。

**首先我们来看下 Promise 示例**，我们从 JavaScript 如何触发、Native 如何回传数据两方面来进行讲解。

首先，JavaScript 端调用客户端定义的 `SystemPropsModule` 的 `getSystemModel` 来获取手机的设备类型，获取结果的方式使用 Promise 方式（`then... catch...`）：

```
NativeModules.SystemPropsModule.getSystemModel().then(result=> {  
  console.log(result);  
}).catch(error=> {  
  console.log(error);  
});
```

然后，Native 端定义 SystemPropsModule，实现 getSystemModel 方法，内部使用 promise 获取手机的 model 数据。使用 promise.resolve(xx) 为成功，promise.reject(xx) 为失败：

```
SystemPropsModule:
...
@ReactMethod
public void getSystemModel(Promise promise) {
    // 回传成功，使用 resolve
    promise.resolve(Build.MODEL);
}
...
```

**接下来看发送事件示例**，我们从 JavaScript 如何监听 Native 事件、Native 如何发送事件这两方面来进行讲解。

首先，JavaScript 端使用 EventEmitterManager 来注册 Native 的事件监听。通过 NativeModules 获取 EventEmitterManager，随后使用它构建出 NativeEventEmitter，最后通过 NativeEventEmitter 注册监听：

```
componentWillMount(){
    // 拿到原生模块
    var eventEmitterManager = NativeModules.EventEmitterManager;
    const nativeEventEmitter = new NativeEventEmitter(eventEmitterManager);
    const eventEmitterManagerEvent = EventEmitterManager.EventEmitterManagerEvent;
    // 监听 Native 发送的通知
    this.listener = nativeEventEmitter.addListener(eventEmitterManagerEvent, (data) =>
        console.log("Receive native event: " + data);
    );
}

componentWillUnmount(){
    // 移除监听
    this.listener.remove();
}
```

在 Native 端的使用则很简单。我们获取 RCTDeviceEventEmitter 这个 JSModule，使用 emit 方法就可以向 JavaScript 发送事件了：

```
reactContext.getJSModule(DeviceEventManagerModule.RCTDeviceEventEmitter.class)
    .emit("msg", "say hello");
```

自定义组件相关的知识点，我们先介绍到这里。接下来进入实战阶段，我们将分别以一个数据存取 TurboModule 和视频播放 Fabric component 的案例，加深你对自定义组件的理解。我们先来看 TurboModule。

## TurboModule：数据存取

TurboModule 采用懒加载模式，在运行时第一次 import 该 TurboModule 时，Native 会创建对应的 TurboModule 并进行缓存。而旧版本的 NativeModule 都是在创建环境时统一进行构造的，会对 React Native 的启动性能有比较大的影响。

接下来我们以一个实际的案例来带你了解 TurboModule。当你需要使用 native 数据存取相关能力，如跨进程存取、偏好存取、加密存取等，而 React Native 自带的数据存储 module 满足不了你的需求，你可以通过自定义数据存储的 TurboModule 来实现。我们先来看下 JavaScript 侧。

## JavaScript

在 Spec 中定义方法，定义好存和取的方法后，再导出 StorageModule：

```
export interface Spec extends TurboModule {
  +save: (key: string, value: string, callback: (value: Object) => void) => void;
  +get: (key: string, callback: (value: Object) => void) => void;
}
// 导出 StorageModule
export default (TurboModuleRegistry.getEnforcing<Spec>(
  'StorageModule',
): Spec);
```

调用：

```
NativeModules.StorageModule.save("testKey", "testValue", (result)=>{
    console.info(result);
});
NativeModules.StorageModule.get("testKey", (result)=>{
    console.info(result);
});
```

接下来我们再看看 Android 端和 iOS 端的实现。

## Android

在实现 StorageModule 之前，我们需要在混合工程中，将 React Native 新架构的运行配置搭建好，这套配置可以运行 TurboModule、Fabric，后面的 Fabric 案例也是基于此配置来运行的。

而且，上一讲在我们已经讲解了如何基于 React Native 最新版本（0.68.0）搭建混合应用，我们再这个基础上开启新架构配置就好了。

**第一步，我们要做些准备工作，也就是获取 newarchitecture 的模版代码。**

我们以 0.68.0 版本创建一个 React Native 工程，来获取新架构的模版代码，我们把这个工程名叫做ReactNativeNewArch：

```
npx react-native init ReactNativeNewArch --version 0.68.0
```

创建好后，你将看到如下工程代码，包含 Java 和 C++：



这些工程代码主要是新架构的 JSI、Fabric、TurboModule 的注册和加载代码，内部逻辑非常复杂，这一讲我们就不做过多分析了，先专注于实操部分。

如果我们想基于这个 Demo 去运行新架构，可以做如下操作：



1. 修改 android 目录下的 gradle.properties:

# 开启新架构

newArchEnabled=true

# 配置 java home 为 JDK 11

org.gradle.java.home=/Library/Java/JavaVirtualMachines/jdk-11.0.2.jdk/Contents/Home

2. 运行

yarn react-native run-android

**第二步，拷贝 newarchitecture 的模版代码到我们之前的混合工程。**

这一步中，我们需要将 Java 层和 C++ 层代码拷贝到混合工程中，需要拷贝的相关代码如下：

Java 层:

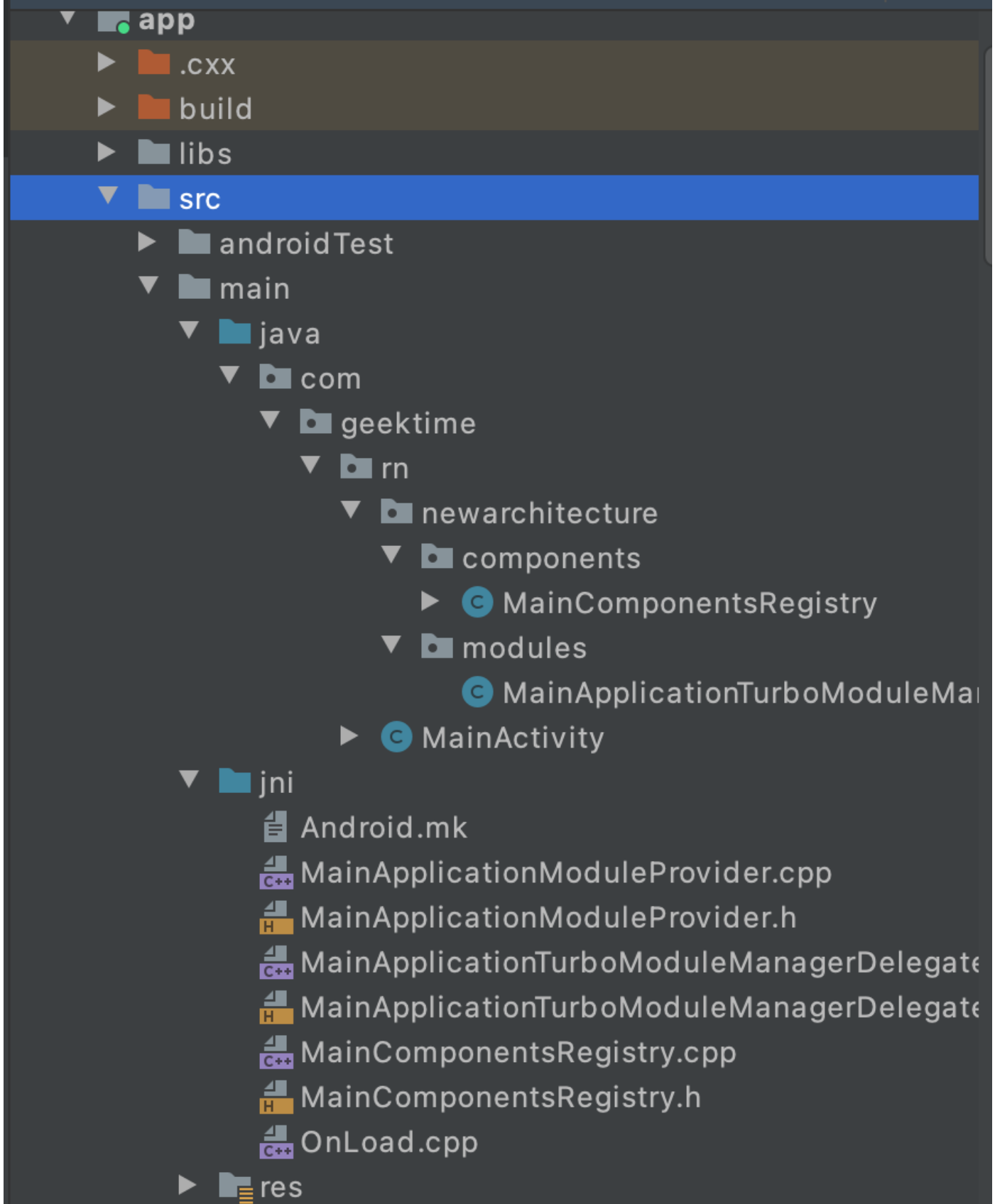
- MainComponentsRegistry.java

- MainApplicationTurboModuleManagerDelegate.java

C++ 层:

- jni 目录

拷贝完的效果是这样的：



第三步是修改拷贝的代码，主要是下面这四点。

1.MainApplicationTurboModuleManagerDelegate.java：

修改 so 库名称为我们自定义名称 `geektime_new_arch`。

```

@Override
protected synchronized void maybeLoadOtherSoLibraries() {
    if (!sIsSoLibraryLoaded) {
        SoLoader.loadLibrary("geektime_new_arch");
        sIsSoLibraryLoaded = true;
    }
}

```

**2.jni/Android.mk:** 修改 Android.mk 中的 so 库名称为上面的 `geektime_new_arch`。

```

# You can customize the name of your application .so file here.
LOCAL_MODULE := geektime_new_arch

```

**3.jni/MainApplicationTurboModuleManagerDelegate.h:** 修改

MainApplicationTurboModuleManagerDelegate 对应的 Java 类路径，截图中拷贝好的 MainApplicationTurboModuleManagerDelegate.java 路径为 `com/reactnativewarch/newarchitecture/modules`。

```

static constexpr auto kJavaDescriptor =
    "Lcom/reactnativewarch/newarchitecture/modules/MainApplicationTurboModuleManagerDelega

```

**4.jni/MainComponentsRegistry.h:** 修改 MainComponentsRegistry 对应的 Java 类路径，截图中拷贝好的 MainComponentsRegistry.java 路径为 `com/reactnativewarch/newarchitecture/components`。

```

constexpr static auto kJavaDescriptor =
    "Lcom/reactnativewarch/newarchitecture/components/MainComponentsRegistry;";

```

做完后，最后一步就是修改 React Native 初始化代码了。

这里有两处要修改。第一处是设置 `ReactPackageTurboModuleManagerDelegateBuilder` 为上面的 `MainApplicationTurboModuleManagerDelegate`（`TurboModule` 用）；第二处是设置 `setJSIModulesPackage`（`Fabric` 用，`JSI` 实现）：

```
public void initRN() {
    ReactInstanceManagerBuilder builder = ReactInstanceManager.builder()
        .setApplication((Application) getApplicationContext())
        .addPackage(new MainReactPackage())
        .setJSMainModulePath("index.android")
        .setInitialLifecycleState(LifecycleState.BEFORE_CREATE)
        .setReactPackageTurboModuleManagerDelegateBuilder(new MainApplicationTurboModuleManagerDelegateBuilder())
        .setJSIModulesPackage(getJSIModulePackage());

    ReactInstanceManager reactInstanceManager = builder.build();
}
```

其中 `getJSIModulePackage()` 我特意摘出来放在了下面。这段代码实现需要从模版代码的 `MainApplicationReactNativeHost` 的 `getJSIModulePackage` 拷贝，内部调用了上面的 `MainComponentsRegistry` 进行注册：

```
static JSIModulePackage getJSIModulePackage() {
    return new JSIModulePackage() {
        @Override
        public List<JSIModuleSpec> getJSIModules(
            final ReactApplicationContext reactApplicationContext,
            final JavaScriptContextHolder jsContext) {
            final List<JSIModuleSpec> specs = new ArrayList<>();
            specs.add(
                new JSIModuleSpec() {
                    @Override
                    public JSIModuleType getJSIModuleType() {
                        return JSIModuleType.UIManager;
                    }

                    @Override
                    public JSIModuleProvider<UIManager> getJSIModuleProvider() {
                        final ComponentFactory componentFactory = new ComponentFactory();
                        CoreComponentsRegistry.register(componentFactory);
                        MainComponentsRegistry.register(componentFactory);

                        List<ViewManager> viewManagers = new ArrayList<>();
                        ViewManagerRegistry viewManagerRegistry = new ViewManagerRegistry(viewManagers);
```

```

        return new FabricJSIModuleProvider(
            reactApplicationContext,
            componentFactory,
            new EmptyReactNativeConfig(),
            viewManagerRegistry);
    }
});
return specs;
}
};
}

```

至此，我们新架构的运行环境就配置好了。由于目前新架构文章非常少，几乎没有混合工程运行新架构的方案，上面这些主要是我们用了大量的时间去调研和测试的结果，你可以参考一下。

好了，回到正题。在混合工程中，新架构的运行环境搭建好后，我们就可以简单快速地来写 TurboModule 和 Fabric 了。

我们继续来进行数据存储的 Demo 在 Java 层定义并实现 StorageModule。Android 端我们使用 SharedPreferences 来实现轻量级偏好存取。这里要注意，你需要继承 ReactModuleWithSpec 和 TurboModule，具体代码如下：

```

// 1. 定义 StorageModule, 继承 ReactContextBaseJavaModule 类
// 实现 ReactModuleWithSpec & TurboModule 接口
public class StorageModule extends ReactContextBaseJavaModule
    implements ReactModuleWithSpec, TurboModule {
    // native 存储的 sp 文件名称
    private static final String SP_NAME = "rn_storage";
    // 返回给 JS 的结果码
    private static final int CODE_SUCCESS = 1;
    private static final int CODE_ERROR = 2;

    public StorageModule(ReactApplicationContextWrapper reactContext) {
        super(reactContext);
    }

    // 返回 module 名称，一般以类名作为 module 名称
    @Override
    public String getName() {
        return StorageModule.class.getSimpleName();
    }
}

```

```

// 定义供 JS 调用的存储数据方法, isBlockingSynchronousMethod 表示是否同步执行
@ReactMethod(isBlockingSynchronousMethod = true)
public void save(String key, String value, Callback callback) {
    WritableMap result = new WritableNativeMap();
    // 如果 js 传入的 key 为空, 则回传失败码和信息
    if (TextUtils.isEmpty(key)) {
        result.putInt("code", CODE_ERROR);
        result.putString("msg", "key is empty or null");
        callback.invoke(result);
        return;
    }
    // 调用 native 的 sp 进行数据存储
    SharedPreferences sp = getReactApplicationContext().getSharedPreferences(SP_NAME, Context.MODE_PRIVATE);
    sp.edit().putString(key, value).apply();
    result.putInt("code", CODE_SUCCESS);
    result.putString("msg", "save success");
    // 回传 js 告知存储成功
    callback.invoke(result);
}

// 定义供 JS 调用的获取数据方法, isBlockingSynchronousMethod 表示是否同步执行
@ReactMethod(isBlockingSynchronousMethod = true)
public void get(String key, Callback callback) {
    // 如果 js 传入的 key 为空, 则回传失败码和信息
    WritableMap result = new WritableNativeMap();
    if (TextUtils.isEmpty(key)) {
        result.putInt("code", CODE_ERROR);
        result.putString("msg", "key is empty or null");
        callback.invoke(result);
        return;
    }
    // 调用 native 的 sp 获取 key 对应的 value 值
    SharedPreferences sp = getReactApplicationContext().getSharedPreferences(SP_NAME, Context.MODE_PRIVATE);
    String value = sp.getString(key, "");
    result.putInt("code", CODE_SUCCESS);
    result.putString("data", value);
    // 将结果回传给 js
    callback.invoke(result);
}
}
}

```

然后是注册组件, 注意 Package 需要继承 TurboReactPackage:

```

public class MyTurboModulePackage extends TurboReactPackage {
    @Override

```

```

    public NativeModule getModule(String name, ReactApplicationContext reactContext) {
        switch(name) {
            case "StorageModule":
                return new StorageModule(reactContext);
                break;
            default:
                return null;
        }
    }

    @Override
    public ReactModuleInfoProvider getReactModuleInfoProvider() {
        //...
    }
}

```

接下来注册到 `ReactInstanceManager` （使用 `reactInstanceManagerBuilder`注册）：

```

ReactInstanceManagerBuilder builder = ReactInstanceManager.builder()
    .addPackage(new MyTurboModulePackage());
... // 其他 RN 初始化配置
ReactInstanceManager reactInstanceManager = builder.build();

```

然后我们利用新架构提供的 `Codegen`，生成新架构需要的 `native` 代码。不过，在使用 `codegen` 之前，我们需要在项目中应用相关的插件：

(1) 在工程根目录安装 `react-native-gradle-plugin`。

```
yarn add react-native-gradle-plugin
```

(2) 在工程根目录的 `settings.gradle` 中配置 `react-native-gradle-plugin`，使用复合构建引入。

```

include ':app'
rootProject.name = "GeekTimeRNAndroid"

```

```
includeBuild('./node_modules/react-native-gradle-plugin')
```

(3) 在 app/build.gradle 中应用插件。

```
apply plugin: "com.facebook.react"
```

这样做后，工程的 gradle 任务中就会出现 generateCodegenArtifactsFromSchema task。

配置好后，我们以后就可以使用 codegen 能力了，然后执行 generateCodegenArtifactsFromSchema，最后运行App

```
../gradlew generateCodegenArtifactsFromSchema
```

Android端的就是这样，现在我们看iOS端需要怎么做。

## iOS

在iOS端中，首先我们要创建一个类并遵循一个协议 spec，协议中包含注册的API声明。而且，该协议需要遵循 RCTBridgeModule 协议和 RCTTurboModule 协议，并且创建一个 JSI。示例代码如下：

```
//定义一个Spec协议
@protocol DataStorageTurboModuleSpec <RCTBridgeModule, RCTTurboModule>
- (NSString *)getString:(NSString *)string;
@end

//JSI实现
namespace facebook {
namespace react {
class JSI_EXPORT DataStorageTurboModuleSpecJSI : public ObjCTurboModule {
public:
    DataStorageTurboModuleSpecJSI(const ObjCTurboModule::InitParams &params);
};
} // namespace react
} // namespace facebook

//定义一些方法
```



```

namespace facebook {
  namespace react {
    static facebook::jsi::Value __hostFunction_DataStorageTurboModuleSpecJSI_getString(
      facebook::jsi::Runtime &rt,
      TurboModule &turboModule,
      const facebook::jsi::Value *args,
      size_t count){
      return static_cast<ObjCTurboModule &>(turboModule)
        .invokeObjCMethod(rt, VoidKind, "getString", @selector(getString:), args, count);
    }
    DataStorageTurboModuleSpecJSI::NativeSampleTurboModuleSpecJSI(const ObjCTurboModule::Init
      : ObjCTurboModule(params)
    {
      //MethodMetadata 第一个参数为0代表该方法有一个参数
      methodMap_["getString"] = MethodMetadata{1, __hostFunction_DataStorageTurboModuleSpecJSI
    }
  }
}

//TurboModule遵循Spec协议
@interface DataStorageTurboModule : NSObject <DataStorageTurboModuleSpec>

@end

```

之后，我们要在该类中注册组件名和API：

```

//DataStorageTurboModule.mm
@implementation DataStorageTurboModule
RCT_EXPORT_MODULE()

- (std::shared_ptr<facebook::react::TurboModule>)getTurboModule:
  (const facebook::react::ObjCTurboModule::InitParams &)params{
  //指定JSI
  return std::make_shared<DataStorageTurboModuleSpecJSI>(params);
}

RCT_EXPORT_METHOD(getString:(NSString *)string){
  NSLog(@"");
}
@end

```

以上便是自定义一个 TurboModule的流程。其实定义 TurboModule 并不复杂，而且 Facebook 也提供了代码生成工具 codegen，比较复杂的是在混合工程中搭建新架构的运行

环境。前面我们花了不少内容讲述如何在客户端开启新架构，接下来的 Fabric 组件介绍也将在新架构环境基础上进行讲解，接下来我们继续来看Fabric 自定义组件。

## Fabric：视频播放

Fabric 对标旧框架的 UIManager。FabricUIManager 可以和 C++ 层直接进行通讯，解除了原有的 UIManager 依赖单个 bridge 的问题。有了 JSI 后，以前批量依赖 bridge 的 UI 操作，都可以同步执行到 C++ 层，性能得到大幅提升，特别是在列表快速滑动、复杂动画交互方面提升更加明显。

现在，我们以一个视频播放组件为例，讲讲如何定义 Fabric 组件。我们先来看下 JavaScript 端的实现。

## JavaScript

JavaScript需要定义属性以及 API，并 export 组件。示例代码如下：

```
type NativeProps = $ReadOnly<{|
  ...ViewProps,
  url?: string
|}>; // 定义视频播放的属性，url 为视频地址

export type VideoViewType = HostComponent<NativeProps>;

// 定义视频播放的方法，包括开始播放、停止播放、暂停播放
interface NativeCommands {
  +callNativeMethodToPlayVideo: (
  ) => void;
  +callNativeMethodToStopVideo: (
  ) => void;
  +callNativeMethodToPauseVideo: (
  ) => void;
}

//导出外部调用的命令，包括开始播放、停止播放、暂停播放
export const Commands: NativeCommands = codegenNativeCommands<NativeCommands>({
  supportedCommands: ['callNativeMethodToPlayVideo'],
  supportedCommands: ['callNativeMethodToStopVideo'],
  supportedCommands: ['callNativeMethodToPauseVideo'],
});

// 导出包装好的组件，其中 VideoView 为引入 Native 的组件
export default (codegenNativeComponent<NativeProps>(<
```

```
    'VideoView',  
  ): VideoViewType);
```

JavaScript 端使用该组件：

```
// 导入 ViewView 组件和工具  
import VideoView, {  
  Commands as VideoViewCommands,  
} from './VideoNativeComponent';  
  
// 外部调用此方法即可调用 ViewView 视频播放能力  
export default function MyView(props: {}): React.Node {  
  return (  
    <View>  
      <VideoView url={"url"} style={{flex: 1}} />  
      <Button title="play" onPress={()=>{  
        VideoViewCommands.callNativeMethodToPlayVideo();  
      }} />  
    </View>  
  )  
}
```

接下来我们再看看 Android 端和 iOS 端的实现。

## Android

由于在前面 TurboModule 的部分，我们已经讲解了如何在混合工程中开启新架构运行模式，我么这里就不再重复了。前面的方法同样适用于 Fabric，我们只需要搭建一次就好了。所以现在要在 Android 端实现 Fabric 组件也非常简单，我们来看下具体实现。

### 第一步，定义视频播放接口。

这里我们要定义 VideoViewManagerInterface，其中包含三个方法：播放视频、停止播放、暂停播放：

```
public interface VideoViewManagerInterface<T extends View> {
    void playVideo(T view, String url);
    void stopVideo(T view);
    void pauseVideo(T view);
}
```

## 第二步，定义视频播放 View。

这一步中，我们要实现视频播放的 View。在 View 中，我们需要实现视频的播放、停止和暂停功能。但播放能力的实现并不是我们讲解的重点，我们这一讲侧重于新架构中 Fabric 组件的实现流程，所以我们这边使用伪代码：

```
public class MyVideoView extends View {
    // ...

    public void playVideo(String url) {
        // 播放视频实现
    }

    public void stopVideo() {
        // 停止视频播放实现
    }

    public void pauseVideo() {
        // 暂停视频播放实现
    }
}
```

## 第三步，定义 ViewManager。

在这一步中，我们要实现暴露给 React Native 调用的能力，包括视频播放、停止，以及暂停，内部会转发到上面我们定义的视频播放 View 的实现中。示例代码如下：

```
@ReactModule(name = VideoViewManager.REACT_CLASS)
public class VideoViewManager: ViewGroupManager<VideoView>(), VideoViewManagerInterface<VideoView> {
    private static final String REACT_CLASS = "VideoView";
```

```

public VideoViewManager() {
}

@Override
public String getName() {
    return REACT_CLASS;
}

@Override
public VideoView createViewInstance(ThemedReactContext reactContext) {
    return new MyVideoView(reactContext);
}

@ReactProp(name = "url")
@Override
public void playVideo(VideoView view, String url) {
    view.playVideo(url);
}

@Override
public void stopVideo(VideoView view) {
    view.stopVideo();
}

@Override
public void pauseVideo(VideoView view) {
    view.pauseVideo();
}
}

```

**最后一步就是注册 `ViewManager`。**我们在 `ReactInstanceManager` 的 `JSIModulesPackage` 中注册 `VideoViewManager`:

```

List<ViewManager> viewManagers = new ArrayList<>();
viewManagers.add(new VideoViewManager())
ViewManagerRegistry viewManagerRegistry = new ViewManagerRegistry(viewManagers);

return new FabricJSIModuleProvider(
    reactApplicationContext,
    componentFactory,
    new EmptyReactNativeConfig(),
    viewManagerRegistry);

```

然后我们利用新架构提供的 Codegen，调用 gradlew generateCodegenArtifactsFromSchema 生成代码 Native 代码：

```
../gradlew generateCodegenArtifactsFromSchema
```

最后，运行即可。Android 端的实现就是这样，接下来我们再看下 iOS 端。

## iOS

在iOS端汇总，首先我们要创建一个继承于RCTViewComponentView 的一个类作为视频组件，如下：

```
@interface VideoComponentView : RCTViewComponentView
//声明播放器组件的一些方法
//播放视频
- (void)playVideo;
//停止视频
- (void)stopVideo;
//暂停视频
- (void)pauseVideo;
@end
```

之后，该类需要遵循一个协议，协议中需要声明执行Command的方法名，示例代码如下：

```
@protocol VideoComponentViewProtocol <NSObject>
- (void)callNativeMethodToPlayVideo;
- (void)callNativeMethodToStopyVideo;
- (void)callNativeMethodToPauseVideo;
@end

RCT_EXTERN inline void VideoComponentCommand(
    id<VideoComponentViewProtocol> componentView,
    NSString const *commandName,
    NSArray const *args)

if([commandName isEqualToString:@"callNativeMethodToPlayVideo"]){
    [componentView callNativeMethodToPlayVideo];
}
```

```

        return;
    }

    if (![commandName isEqualToString:@"callNativeMethodToStopVideo"]){
        [componentView callNativeMethodToStopVideo];
        return;
    }

    if (![commandName isEqualToString:@"callNativeMethodToPauseVideo"]){
        [componentView callNativeMethodToPauseVideo];
        return;
    }
    return;
}
)

```

接下来，ComponentView需要遵循该Protocol协议，并在执行common时调用对应的方法。此外，我们还可以设置组件的属性：

```

using namespace facebook::react;

@interface VideoComponentView() <VideoComponentViewProtocol>
@end

@implementation VideoComponentView{
    VideoPlayer *_videoPlayer;
}

#pragma mark - Native Commands
- (void)handleCommand:(const NSString *)commandName args:(const NSArray *)args{
    VideoComponentCommand(self, commandName, args);
}

- (void)callNativeMethodToPlayVideo{
    //实现视频播放功能
    [_videoPlayer startPlay];
}

- (void)callNativeMethodToStopVideo{
    //实现视频停止功能
    [_videoPlayer stopPlay];
}

- (void)callNativeMethodToPauseVideo{
    //实现视频暂停功能
    [_videoPlayer pausePlay];
}

```

```

}

#pragma mark - Props
//遵循descriptor协议
+ (ComponentDescriptorProvider)componentDescriptorProvider{
    return concreteComponentDescriptorProvider<VideoComponentDescriptor>();
}

- (instancetype)initWithFrame:(CGRect)frame{
    if (self = [super initWithFrame:frame]) {
        static const auto defaultProps = std::make_shared<const ComponentViewProps>();
        _props = defaultProps;

        _videoPlayer = [[VideoPlayer alloc] init];
        self.contentView = _videoPlayer;
    }
    return self;
}

- (void)updateProps:(Props::Shared const &)props oldProps:(Props::Shared const &)oldProps{
    [super updateProps:props oldProps:oldProps];
}

- (void)onChange:(UIView *)sender{
    // No-op
    // std::dynamic_pointer_cast<const ViewEventEmitter>(_eventEmitter)
    //      ->onChange(ViewEventEmitter::OnChange{.value = static_cast<bool>(sender.on)});
}

@end

Class<RCTComponentViewProtocol> VideoViewCls(void){
    return VideoComponentView.class;
}

```

接着，注册属性遵循VideoComponentDescriptor，并且需要指定该组件的名字：

```

namespace facebook {
namespace react {
using VideoComponentDescriptor = ConcreteComponentDescriptor<VideoViewShadowNode>;
} // namespace react
} // namespace facebook

namespace facebook {
namespace react {
extern const char VideoViewComponentName[];

```



```

using VideoViewShadowNode = ConcreteViewShadowNode<
    VideoViewComponentName,//组件名
    VideoViewProps>;//注册属性
} // namespace react
} // namespace facebook

namespace facebook {
namespace react {
extern const char VideoViewComponentName[] = "VideoView";//组件名
} // namespace react
} // namespace facebook

namespace facebook {
namespace react {
//属性定义
class VideoViewProps final : public ViewProps {
public:
    VideoViewProps() = default;
    VideoViewProps(const PropsParserContext& context, const VideoViewProps &sourceProps, const

    #pragma mark - Props
    std::string url{""};//视频url
};
} // namespace react
} // namespace facebook

```

这样，我们就创建好了一个 React Native 的 Fabric 组件、定义属性以及 API 的方法。以上便是如何使用 Fabric 自定义视频播放组件，在混合工程中搭建好新架构的运行环境后，只需要遵守 Fabric 的组件定义方式，进行接口定义、功能实现和组件注册即可。关于一些复杂的 Fabric 组件，可以查看 <https://github.com/software-mansion/react-native-gesture-handler/tree/main/FabricExample>，<https://github.com/software-mansion/react-native-reanimated/tree/main/FabricExample>，目前 react-native-gesture-handler、react-native-reanimated 都已经适配了新架构，感兴趣的同学可以去学习下。

## 总结

这一讲，我们系统讲解了个性化组件的使用场景、生命周期、传输类型，以及通信方式，并通过两个实际案例讲解了如何在新架构下定制个性化的 TurboModules 与 Fabric。而且，我们也简单介绍了一下 React Native 新架构，你可以通过官方文档进行新架构的体验。

这一讲是我们 Native 相关的三讲中花的时间最长的，也是最“伤肝”的，我们前前后后加调研花了两个月的时间。但我们相信，新架构在未来会有很好的发展，这是可以预见的。因为它解决了 React Native 几个最痛的点，包括启动速度、运行时性能等。如果新架构还能在易用性上继续优化，将会大大拓展 React Native 的用户群体。

因为目前新架构还处于未发布的状态，网上相关的文章大都是对官方纯 React Native 模式 Demo 和介绍，少数几篇会深挖原理，但讲混合模式的新架构运行文章几乎没有。我们这一讲中对 TurboModule 和 Fabric 的讲解，更侧重于如何在混合工程中开启并运行。如果有对 TurboModule、Fabric、JSI 的原理感兴趣的同学，后面有机会我们再来分享。

## 作业

1. 设计一个打印 Native 日志的 TurboModule，以及一个 Native 加载进度条的 Fabric 组件。

欢迎在评论区写下你的想法和经验，和我们多多交流。我们下一讲见。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 21 | 混合应用：如何从零开始集成 React Native?

下一篇 23 | 热更新：如何搭建一个热更新平台?

## 精选留言 2